MIT 8103 Advanced Database Systems

IsaacMart database portfolio summary

Idowu Isaac | Student ID 30168852 | September Cohort 2026

Preparation status

Assistant-prepared and technically tested draft for student review. The repository URL has been supplied. Assessor access, student verification and personal reflection remain pending.

Case study and scope

IsaacMart is a simulated Nigerian e-commerce retailer. Customers order products, warehouse staff manage stock, finance staff reconcile payments and managers analyse sales. PostgreSQL stores authoritative transactions in NGN; MongoDB holds flexible catalogue attributes. All records are synthetic. Delivery, taxes and refunds are outside this exercise.

Portfolio 1 Database design and modelling

The relational model has Customers, Categories, Products, Orders, OrderItems and Payments. Customer-to-order, category-to-product and order-to-payment relationships are one-to-many; OrderItems resolves the order/product many-to-many relationship. Primary keys, foreign keys, unique email/SKU/payment references and CHECK constraints protect row-level integrity. The model is in 3NF under the stated dependencies. Historical unit_price belongs to the sale line; current price belongs to Products. Totals are derived. Cross-row payment reconciliation still requires application checks.

Evidence and data

The design folder contains executable schema/seed SQL, the ER diagram and setup output. The generated dataset contains 10,000 customers, 1,000 products, 200,000 orders, 200,000 line items and 180,000 successful payments. Detailed rules and exclusions appear in the activity README.

Portfolio 2 Query processing and optimisation

The customer-history query filters customer_id and returns the latest ten orders. A B-tree on (customer_id, ordered_at DESC) supports filtering and ordering. Median server execution time across five measured runs after one warm-up was 12.802 ms before and 0.093 ms after indexing (137.66x observed ratio). Raw JSON plans and timings are retained. Cache state, sequential conditions and one local container limit generalisation; index storage and write costs remain. Other queries cover category sales, monthly sales, customers without orders and payment reconciliation.

Portfolio 3 Transactions and concurrency

A committed stock decrement changes 100 to 98; a later five-unit decrement is rolled back, retaining 98. A separate checkout demonstration groups stock reservation, order, line and pending payment, then rolls back. External payment settlement requires idempotency and an outbox/saga; the database cannot atomically commit a remote gateway call.

Two actual PostgreSQL sessions demonstrate row locking: session A holds a product update while B reaches a 500ms lock timeout. A rolls back. A conditional UPDATE with stock >= requested quantity prevents stale read-then-write overselling when callers check the returned row count. REPEATABLE READ evidence shows session A reading 100 twice while B commits 101; after A commits, its new query reads 101. READ COMMITTED instead obtains a fresh statement snapshot. Deadlock/serialization failures require retries.

Portfolio 4 NoSQL and advanced models

MongoDB embeds category-specific attributes in product documents: headphones have battery hours, shirts have size/material, and lamps have watts. The script inserts three documents, queries battery hours, updates an attribute and verifies unique-SKU and schema-validation rejection. PostgreSQL owns price and stock; MongoDB owns descriptive attributes. An outbox-based synchronisation design is discussed but not implemented. No cross-store foreign-key or transaction guarantee is claimed.

Portfolio 5 Distributed and cloud databases

The discussion proposes a managed PostgreSQL primary with a cross-zone standby and asynchronous reporting replicas, alongside a MongoDB replica set. Synchronous acknowledgements trade latency/write availability for reduced loss exposure; asynchronous replicas may lag. Fenced failover must prevent split-brain writers. Browse descriptions may tolerate staleness, while checkout reads authoritative stock. Replication does not replace encrypted backups and restoration drills.

Customer-based sharding could co-locate orders but complicates shared product inventory and cross-shard checkout. Date partitioning supports retention and does not by itself provide multi-server sharding. Optimisation and scale-up should precede sharding. This component follows the brief's discussion option: no cloud deployment, replication, failover or recovery measurements are claimed.

Verification evidence

Evidence consists of genuine assistant-run PostgreSQL/MongoDB text output and JSON plans, rather than fabricated screenshots. Activity READMEs document implementation, testing and limitations. Rerun the scripts and retain your own verification before submission.

Repository and submission review

Repository URL: https://github.com/LaProfessor/Idowu_Isaac_30168852_MIT8103 . Repository contents and assessor access have not been independently verified because the connected GitHub account receives a 404 response. Confirm access before submission. Record future development progressively with honest commit dates.

Individual technical reflection

STUDENT TO COMPLETE AFTER REVIEW: Explain one design choice, one challenge you personally encountered, how you solved it, and one lesson from your own reruns. Discuss why sale-time price is stored, how the composite index changes the plan, and what the concurrency results mean. Replace this paragraph with your authentic reflection; do not submit it as a completed reflection.

AI use declaration

OpenAI ChatGPT/Codex assisted with case-study selection, schema/scripts, synthetic data, diagram, technical explanations, documentation and automated execution in a managed environment. Evidence here was produced by assistant-run containers. Student verification is pending: before submission, record the commands you personally reran, corrections made and how you verified the results. Do not claim independent testing that has not occurred.

References

Miva University, MIT 8103 Advanced Database Systems CA September Cohort 2026, supplied brief.

PostgreSQL 17 documentation: constraints, EXPLAIN, multicolumn indexes, transaction isolation, explicit locking and warm standby (postgresql.org/docs/17/).

MongoDB documentation: data modelling, schema validation and replication (mongodb.com/docs/manual/